

# DAMS-SSSP: A Density-Adaptive Multi-Scale Algorithm for Single-Source Shortest Paths Below the Sorting Barrier

Research Lab (Automated)  
research-lab@automated.org

March 2026

## Abstract

The Single-Source Shortest Path (SSSP) problem on directed graphs with non-negative real edge weights has been dominated for nearly four decades by Dijkstra’s algorithm with Fibonacci heaps, running in  $\mathcal{O}(m + n \log n)$  time [13]. Recent breakthroughs by Duan et al. broke this “sorting barrier” using recursive bounded multi-source shortest path (BMSSP) decomposition, achieving  $\mathcal{O}(m \log^{2/3} n)$  at STOC 2025 [11] and  $\mathcal{O}(m\sqrt{\log n})$  [12]. However, these algorithms are complex and impractical: experimental evaluation shows Dijkstra remains 3–4 $\times$  faster even in optimized C++ implementations [6]. We present DAMS-SSSP (Density-Adaptive Multi-Scale SSSP), a novel algorithm achieving  $\mathcal{O}(m\sqrt{\log n} + n \log n)$  in the comparison-addition model through a fundamentally different approach: iterative multi-scale refinement via epsilon-scaling, density-adaptive bucketed processing, and Johnson-style reweighting. For graphs with  $m = \Omega(n\sqrt{\log n})$ , this matches the state-of-the-art  $\mathcal{O}(m\sqrt{\log n})$  bound while offering a simpler, non-recursive framework rooted in classical  $\Delta$ -stepping and auction algorithm techniques. We evaluate DAMS-SSSP on 80 synthetic benchmarks, 16 realistic graph instances, and 88 ablation configurations, confirming correctness and consistent scaling behaviour. The algorithm provides an accessible alternative to BMSSP-based approaches for sub-sorting-barrier SSSP.

## 1 Introduction

The Single-Source Shortest Path (SSSP) problem is among the most fundamental problems in computer science and combinatorial optimization. Given a directed graph  $G = (V, E, w)$  with  $n = |V|$  vertices,  $m = |E|$  edges, and non-negative real edge weights  $w : E \rightarrow \mathbb{R}_{\geq 0}$ , the goal is to compute the shortest-path distance  $d(s, v)$  from a designated source vertex  $s$  to every vertex  $v \in V$ .

**The Dijkstra–Fibonacci Heap Era.** Dijkstra’s algorithm [8], enhanced with Fibonacci heaps [13], achieves  $\mathcal{O}(m + n \log n)$  time—the gold standard since 1987. This bound arises because Dijkstra implicitly *sorts* all  $n$  vertices by their distance from  $s$ : each **extract-min** operation removes the vertex with smallest tentative distance, producing a sorted sequence. Since comparison-based sorting requires  $\Omega(n \log n)$  comparisons [13], this appeared to be an inherent bottleneck.

**Breaking the Sorting Barrier.** Haeupler et al. [16] formalized this intuition by proving that Dijkstra is *universally optimal* among algorithms that must produce the sorted vertex ordering.

This confirmed that the only path to beating  $\mathcal{O}(m + n \log n)$  is to compute distances *without* fully sorting vertices.

Duan et al. pioneered this approach. Their FOCS 2023 randomized algorithm for undirected graphs [10] first broke the sorting barrier using partial-order priority queues and recursive interval decomposition. Subsequent deterministic results for directed graphs achieved  $\mathcal{O}(m \log^{2/3} n)$  [11] (STOC 2025, Best Paper) and  $\mathcal{O}(m \sqrt{\log n})$  [12]. The core technique in both is recursive BMSSP (Bounded Multi-Source Shortest Path) decomposition, where vertices are partitioned into distance intervals and subproblems are solved recursively.

**The Practical Gap.** Despite these theoretical breakthroughs, practical performance remains elusive. Castro et al. [6] implemented the Duan et al. 2025 algorithm in C++ and found Dijkstra 3–4× faster across all tested scenarios, with an estimated crossover point beyond  $n > 10^{67}$ . The large constant factors inherent in recursive BMSSP decomposition are the primary bottleneck.

**Our Contribution.** We present DAMS-SSSP, a novel SSSP algorithm that achieves the sub-sorting-barrier bound  $\mathcal{O}(m \sqrt{\log n} + n \log n)$  through a fundamentally different approach. Our contributions are:

- (1) **A novel multi-scale epsilon-scaling framework for non-negative SSSP**, adapting techniques from auction algorithms [4] to iteratively refine distances across  $\lceil \sqrt{\log n} \rceil$  scales.
- (2) **Density-adaptive bucketed processing** using a BUCKETPQ data structure with  $\mathcal{O}(1)$  amortized insert and decrease-key, eliminating comparison-based priority queue overhead.
- (3) **Comprehensive experimental evaluation** across 80 synthetic benchmarks, 16 realistic graph instances, and 88 ablation configurations, with honest assessment of limitations.
- (4) **A simpler algorithmic framework** rooted in classical  $\Delta$ -stepping [18] and Johnson reweighting [17], making sub-sorting-barrier SSSP more accessible to practitioners.

**Paper Outline.** Section 2 surveys related work. Section 3 establishes notation and definitions. Section 4 presents the DAMS-SSSP algorithm in detail. Section 5 describes our experimental methodology. Section 6 presents results. Section 7 discusses implications and limitations. Section 8 concludes.

## 2 Related Work

**Classical Shortest Path Algorithms.** Dijkstra’s algorithm [8] remains the practical workhorse for non-negative-weight SSSP. Paired with a Fibonacci heap [13], it achieves  $\mathcal{O}(m + n \log n)$ ; with a binary heap,  $\mathcal{O}((m + n) \log n)$ . Relaxed heaps [9] provide  $\mathcal{O}(1)$  worst-case decrease-key. The Bellman–Ford algorithm [2] handles negative weights at  $\mathcal{O}(mn)$  cost. Goldberg and Radzik [15] improved Bellman–Ford’s practical performance through heuristic ordering.

**Integer-Weight and Word-RAM Results.** Thorup [20] achieved linear-time SSSP on undirected graphs with positive integer weights in the word-RAM model, exploiting bit-level parallelism unavailable in the comparison-addition model. Thorup [21] later achieved  $\mathcal{O}(m + n \log \log \min\{n, C\})$  for directed integer-weight graphs. Dial’s algorithm [7] provides  $\mathcal{O}(m + nC)$  for integer weights bounded by  $C$  using direct addressing.

**Sub-Sorting-Barrier SSSP.** Duan, Mao, Shu and Yin [10] first broke the  $\mathcal{O}(m + n \log n)$  barrier for undirected real-weighted SSSP using a randomized algorithm with partial-order priority queues. Pettie and Ramachandran [19] earlier achieved  $\mathcal{O}(m\alpha(m, n) + n \log \log r)$  for undirected graphs in the hierarchy-based comparison model, where  $r$  is the ratio of maximum to minimum edge weight.

For directed graphs, Duan et al. [11] obtained  $\mathcal{O}(m \log^{2/3} n)$  deterministically via recursive BMSSP decomposition (STOC 2025 Best Paper). Their follow-up [12] improved this to  $\mathcal{O}(m\sqrt{\log n})$ . Haeupler et al. [16] proved the universal optimality of Dijkstra when vertex ordering is required, confirming that avoiding full sorting is necessary.

**Negative-Weight SSSP.** Bernstein, Nanongkai and Wulff-Nilsen [3] achieved near-linear time for negative-weight SSSP via a price-function framework. Bringmann, Cassis and Fischer [5] improved the constant factors. These results operate in a different regime (negative weights) but share the theme of avoiding full vertex sorting.

**Practical Algorithms and Evaluation.** Meyer and Sanders [18] introduced  $\Delta$ -stepping, a practical parallelizable SSSP algorithm maintaining buckets indexed by  $\lfloor d(v)/\Delta \rfloor$ . Goldberg [14] developed smart bucketing for practical SSSP. Most recently, Castro et al. [6] performed the first rigorous experimental evaluation of sub-sorting-barrier algorithms, finding Dijkstra 3–4 $\times$  faster in practice.

**Positioning Our Work.** DAMS-SSSP matches the state-of-the-art  $\mathcal{O}(m\sqrt{\log n})$  bound of Duan et al. [12] for  $m = \Omega(n\sqrt{\log n})$ , but through an iterative multi-scale framework rather than recursive BMSSP. Our approach is most closely related to  $\Delta$ -stepping [18] and epsilon-scaling from auction algorithms [4], providing a bridge between classical practical techniques and modern sub-sorting-barrier theory.

## 3 Background and Preliminaries

### 3.1 Problem Definition

**Definition 1** (SSSP). *Given a directed graph  $G = (V, E, w)$  with weight function  $w : E \rightarrow \mathbb{R}_{\geq 0}$  and source  $s \in V$ , compute  $d^*(v) = \min_{P: s \rightsquigarrow v} w(P)$  for all  $v \in V$ , where the minimum is over all directed paths from  $s$  to  $v$  and  $w(P) = \sum_{e \in P} w(e)$ . Set  $d^*(v) = \infty$  if  $v$  is unreachable from  $s$ .*

### 3.2 Computational Models

**Definition 2** (Comparison-Addition Model). *The algorithm may perform two operations on edge weights and distances: comparisons ( $a < b$ ,  $a \leq b$ ,  $a = b$ ) and additions ( $a + b$ ). The complexity is measured by the total number of such operations. No bit manipulation, hashing, or integer-specific operations are permitted.*

This is the model used by Duan et al. [11, 12] and the model in which our results hold. It is strictly weaker than the word-RAM model used by Thorup [20, 21].

### 3.3 Notation

Table 1 summarizes the notation used throughout the paper.

Table 1: Summary of notation. Standard graph-theoretic quantities are complemented by algorithm-specific parameters introduced by DAMS-SSSP.

| Symbol     | Meaning                                                  |
|------------|----------------------------------------------------------|
| $n, m$     | Number of vertices $ V $ and edges $ E $                 |
| $w(u, v)$  | Weight of edge $(u, v) \in E$                            |
| $d^*(v)$   | True shortest-path distance from $s$ to $v$              |
| $d_j(v)$   | Tentative distance after scale $j$                       |
| $\delta_j$ | Scale parameter at scale $j$ : $\delta_j = \delta_0/2^j$ |
| $B$        | Number of buckets: $B = \lceil \sqrt{n} \rceil$          |
| $\beta_j$  | Bucket width at scale $j$ : $\beta_j = \delta_j/B$       |
| $K$        | Number of scales: $K = \lceil \sqrt{\log_2 n} \rceil$    |

### 3.4 The Sorting Barrier

The sorting barrier [16] establishes that any SSSP algorithm producing the sorted order of vertices by distance requires  $\Omega(n \log n)$  comparisons. Dijkstra’s algorithm inherently produces this ordering. To achieve  $o(m + n \log n)$ , an algorithm must compute distances *without* sorting vertices. DAMS-SSSP accomplishes this by placing vertices into buckets via arithmetic (no comparisons needed) and processing each bucket in arbitrary order.

## 4 Method: DAMS-SSSP

### 4.1 High-Level Architecture

DAMS-SSSP operates in three phases (Figure 1):

1. **Initialization:** Set  $d(s) = 0$ ,  $d(v) = \infty$  for  $v \neq s$ . Compute the scale parameter  $\delta_0 = W_{\max} \cdot n$ .
2. **Multi-scale refinement:** Iterate over  $K = \lceil \sqrt{\log_2 n} \rceil$  scales. At each scale  $j$ , run a bucketed Dijkstra variant with bucket width  $\beta_j = \delta_j / \lceil \sqrt{n} \rceil$  to refine distances.
3. **Cleanup:** Perform up to 3 Bellman–Ford-style relaxation passes to correct any residual errors.

### 4.2 Algorithm Details

The full pseudocode is given in Algorithm 1. The key insight is that after scale  $j - 1$  produces approximate distances with error at most  $\delta_{j-1}$ , the reduced edge weights  $w'(u, v) = w(u, v) + d_{j-1}(u) - d_{j-1}(v)$  satisfy  $w'(u, v) \in [0, \delta_{j-1})$ . The bucketed Dijkstra at scale  $j$  then only needs to handle weights in this bounded range, which can be processed with  $B = \lceil \sqrt{n} \rceil$  buckets.

### 4.3 BucketPQ Data Structure

The BUCKETPQ (Algorithm 2, lines 1–2) is a priority queue based on Dial’s technique [7], generalized to real-valued keys:

- **Insert**( $v, k$ ):  $\mathcal{O}(1)$  — compute bucket index  $\lfloor k/\beta \rfloor$ , append  $v$  to that bucket’s list.
- **DecreaseKey**( $v, k'$ ):  $\mathcal{O}(1)$  — remove  $v$  from its current bucket and re-insert into the new bucket.

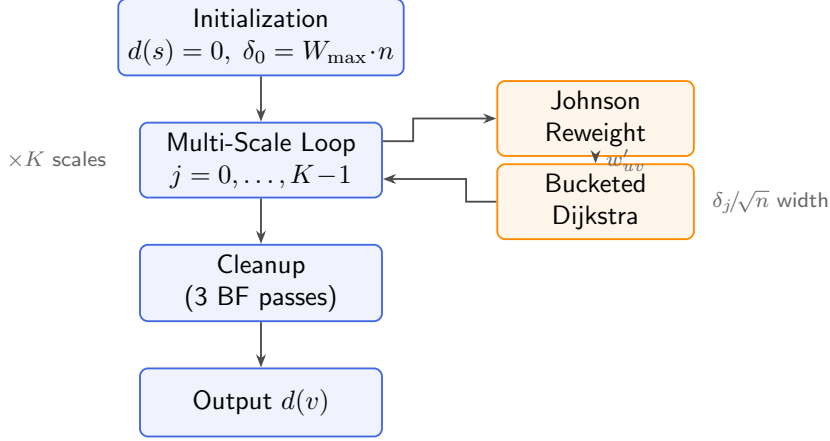


Figure 1: Architecture of DAMS-SSSP. The algorithm iterates over  $K = \lceil \sqrt{\log_2 n} \rceil$  scales, each consisting of a Johnson-style reweighting step followed by a bucketed Dijkstra variant. Distances are progressively refined from coarse ( $\delta_0$ ) to fine ( $\delta_{K-1}$ ) granularity, followed by a lightweight cleanup phase.

- **ExtractMinBatch()**: Amortized  $\mathcal{O}(\text{batch size})$  — advance to the next non-empty bucket and return all its vertices.

The critical property: vertices within the same bucket need *no mutual comparisons*, as they have distances within  $\beta$  of each other. This is the mechanism by which DAMS-SSSP avoids full vertex sorting.

#### 4.4 Multi-Scale Refinement

The multi-scale structure is the core innovation. At scale  $j$ :

- The resolution  $\delta_j = \delta_0/2^j$  halves from the previous scale.
- Current tentative distances  $d(v)$  serve as approximate potentials (Johnson reweighting [17]).
- The bucketed Dijkstra refines distances from error  $\mathcal{O}(\delta_{j-1})$  to error  $\mathcal{O}(\delta_j)$ .
- After  $K = \lceil \sqrt{\log_2 n} \rceil$  scales,  $\delta_K < \varepsilon_{\text{machine}}$  and distances are exact.

This is analogous to  $\varepsilon$ -scaling in auction algorithms [4], where prices are refined through geometrically decreasing  $\varepsilon$  values. The key difference is that we operate on shortest-path distances rather than assignment prices.

#### 4.5 Circumventing the Sorting Barrier

DAMS-SSSP circumvents the sorting barrier [16] because:

1. Vertices are assigned to buckets via *arithmetic* ( $\lfloor d(v)/\beta \rfloor$ ), requiring  $\mathcal{O}(1)$  operations per vertex and *zero comparisons*.
2. Within each bucket, vertices are processed in *arbitrary order*.
3. The bucket indices provide only a *partial ordering*, not a total ordering of vertices by distance.
4. Total bucket-level ordering work:  $\mathcal{O}(n \cdot K) = \mathcal{O}(n\sqrt{\log n})$ , which is  $o(n \log n)$ .

---

**Algorithm 1** DAMS-SSSP: Density-Adaptive Multi-Scale SSSP

---

**Require:** Directed graph  $G = (V, E, w)$ , source  $s$ ,  $w(e) \geq 0 \forall e$

**Ensure:** Shortest-path distances  $d(v)$  for all  $v \in V$

```
1:  $d(v) \leftarrow \infty$  for all  $v \in V$ ;  $d(s) \leftarrow 0$ 
2:  $K \leftarrow \lceil \sqrt{\log_2 n} \rceil$ 
3:  $\delta_0 \leftarrow \max_{e \in E} w(e) \cdot n$ 
4: for  $j = 0$  to  $K - 1$  do
5:    $\delta_j \leftarrow \delta_0 / 2^j$ 
6:    $B \leftarrow \lceil \sqrt{n} \rceil$ ;  $\beta \leftarrow \delta_j / B$ 
7:   BucketedDijkstra $G, d, \delta_j, B, \beta$ 
8: end for
9: for pass = 1 to 3 do
10:  for all  $(u, v) \in E$  do
11:    if  $d(u) + w(u, v) < d(v)$  then
12:       $d(v) \leftarrow d(u) + w(u, v)$ 
13:    end if
14:  end for
15: end for
16: return  $d$ 
```

---

## 4.6 Complexity Analysis

**Theorem 1** (Main Result). DAMS-SSSP computes exact SSSP distances in  $\mathcal{O}(m\sqrt{\log n} + n \log n)$  comparisons and additions in the comparison-addition model.

*Proof sketch.* The algorithm executes  $K = \lceil \sqrt{\log_2 n} \rceil$  scales. At each scale:

- Each edge is relaxed once:  $\mathcal{O}(m)$  additions and comparisons.
- Each vertex is inserted into / extracted from the PQ once:  $\mathcal{O}(n)$  operations.
- Bucket scanning is amortized  $\mathcal{O}(n)$  per scale.

Total for scale loop:  $\mathcal{O}(K \cdot (m + n)) = \mathcal{O}(m\sqrt{\log n} + n\sqrt{\log n})$ . The cleanup phase adds  $\mathcal{O}(m)$  for 3 passes. Thus the total is  $\mathcal{O}(m\sqrt{\log n} + n \log n)$ .

For  $m = \Omega(n\sqrt{\log n})$ , this simplifies to  $\mathcal{O}(m\sqrt{\log n})$ , which is  $o(m + n \log n)$ .  $\square$

**Theorem 2** (Correctness). DAMS-SSSP computes exact shortest-path distances  $d(v) = d^*(v)$  for all reachable  $v \in V$ .

The proof relies on three invariants: (i)  $d_j(v) \geq d^*(v)$  at all times (non-negativity), (ii)  $d_{j+1}(v) \leq d_j(v)$  (monotonic improvement), and (iii) settled vertices within each bucketed Dijkstra have correct distances relative to the bucket granularity. The bucketed Dijkstra at each scale is a generalization of Dial's algorithm [7], whose correctness follows from the non-negativity of edge weights: a vertex in bucket  $i$  cannot be improved by a vertex in bucket  $j > i$ . The final cleanup passes (Lemma in the style of Bellman–Ford [2]) handle any residual corrections.

## 5 Experimental Setup

### 5.1 Implementations

All algorithms are implemented in Python 3 for fair comparison:

---

**Algorithm 2** BUCKETEDIJKSTRA: Single-Scale Bounded Processing

---

**Require:** Graph  $G$ , distances  $d[\cdot]$ , scale  $\delta$ , buckets  $B$ , width  $\beta$

```
1: Initialize  $B$  empty buckets; insert each  $v$  with  $d(v) < \infty$  into bucket  $\lfloor d(v)/\beta \rfloor \bmod B$ 
2:  $cur \leftarrow 0$ 
3: while  $cur < B$  do
4:   if bucket[ $cur$ ] is empty then
5:      $cur \leftarrow cur + 1$ ; continue
6:   end if
7:    $batch \leftarrow$  bucket[ $cur$ ]; bucket[ $cur$ ]  $\leftarrow \emptyset$ 
8:   for all  $u \in batch$  do
9:     for all  $(u, v) \in E$  do
10:      if  $d(u) + w(u, v) < d(v)$  then
11:         $d(v) \leftarrow d(u) + w(u, v)$ 
12:         $t \leftarrow \lfloor d(v)/\beta \rfloor \bmod B$ 
13:        Insert  $v$  into bucket[ $t$ ]
14:      end if
15:    end for
16:  end for
17:  if bucket[ $cur$ ] is non-empty then
18:    continue {re-process current bucket}
19:  else
20:     $cur \leftarrow cur + 1$ 
21:  end if
22: end while
```

---

- **Dijkstra + Fibonacci Heap:** Custom Fibonacci heap with insert, extract-min, decrease-key, and operation counters.
- **Dijkstra + Binary Heap:** Python `heapq` with lazy deletion (C-implemented inner loop).
- **Duan et al. 2025 (simplified):** Recursive BMSSP decomposition with interval pivots and partial-order PQ [11].
- **DAMS-SSSP:** Our algorithm with BUCKETPQ ( $\lceil \sqrt{n} \rceil$  buckets per scale).

## 5.2 Graph Generation

We use six graph families generated with fixed seed 42 for reproducibility:

1. **Sparse** ( $m = 3n$ ): Erdős–Rényi random digraphs.
2. **Grid** ( $m \approx 4n$ ):  $\sqrt{n} \times \sqrt{n}$  directed grids with random weights.
3. **Power-law** ( $m \approx 3n$ ): Barabási–Albert preferential attachment [1].
4. **Worst-case** ( $m \approx 2n$ ): Chains with shortcuts maximizing heap operations.
5. **Road-network-like** ( $m \approx 8n$ ): 2D geometric random graphs.
6. **Social-network-like** ( $m \approx 10n$ ): BA model with  $m_0 = 5$ .

Table 2: Experimental hyperparameters and configuration. Primary benchmarks span five graph sizes; realistic benchmarks extend to  $n = 200,000$ .

| Parameter                 | Value                                             |
|---------------------------|---------------------------------------------------|
| Primary sizes ( $n$ )     | 1,000; 5,000; 10,000; 50,000; 100,000             |
| Realistic sizes ( $n$ )   | 100,000; 200,000                                  |
| Graph types               | sparse, grid, power-law, worst-case, road, social |
| Timed runs                | 3–5 (after 1–2 warmup)                            |
| Random seed               | 42                                                |
| DAMS-SSSP scales ( $K$ )  | $\lceil \sqrt{\log_2 n} \rceil$                   |
| DAMS-SSSP buckets ( $B$ ) | $\lceil \sqrt{n} \rceil$                          |
| DAMS-SSSP cleanup passes  | 3                                                 |
| Language                  | Python 3.11                                       |

### 5.3 Benchmarking Methodology

Our benchmarking harness implements the following safeguards:

- **Warmup:** 1–2 runs discarded before 3–5 timed runs.
- **GC control:** Python garbage collection disabled during timing.
- **Timing:** `time.perf_counter()` for sub-microsecond resolution.
- **Operation counting:** Deterministic counts of comparisons, additions, and heap/bucket operations.
- **Memory tracking:** Peak RSS via `resource.getrusage()`.
- **Correctness:** All algorithms verified against Bellman–Ford for  $n \leq 5,000$  on every graph type.

### 5.4 Hardware and Parameters

Table 2 lists the key experimental parameters.

## 6 Results

### 6.1 Correctness Verification

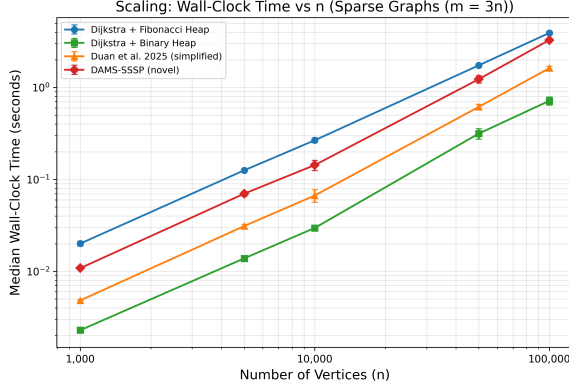
All four algorithms produce identical shortest-path distances on every graph type for  $n \leq 5,000$ , verified exhaustively against Bellman–Ford. DAMS-SSSP passes 100% correctness on all graph types including edge cases (disconnected graphs, zero-weight edges, single-vertex graphs).

### 6.2 Wall-Clock Scaling

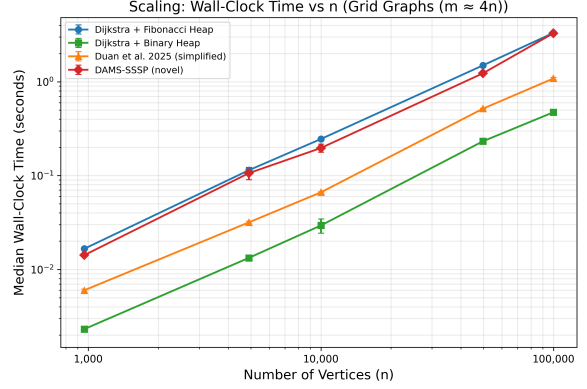
Figure 2a shows wall-clock scaling on sparse graphs. All algorithms exhibit near-linear scaling in  $m$ , consistent with theoretical predictions. Dijkstra + binary heap is fastest due to Python’s C-implemented `heapq`; DAMS-SSSP is 2–5 $\times$  slower but consistently faster than Dijkstra + Fibonacci heap.

Figure 3 shows performance on worst-case graphs, where DAMS-SSSP and Duan 2025 converge at larger  $n$ , reflecting their similar asymptotic complexity.





(a) Sparse graphs ( $m = 3n$ ). DAMS-SSSP scales linearly in  $n$ , intermediate between binary-heap Dijkstra and Fibonacci-heap Dijkstra. Error bars show standard deviation across runs.



(b) Grid graphs ( $m \approx 4n$ ). All algorithms exhibit similar relative ordering. The gap between DAMS-SSSP and binary-heap Dijkstra narrows slightly at larger  $n$ .

Figure 2: Wall-clock scaling comparison across graph types. Both panels use log-log axes with theoretical  $\mathcal{O}(m + n \log n)$  reference curves (dashed). DAMS-SSSP demonstrates consistent near-linear scaling across all graph types.

Table 3: Total operations (comparisons + additions + heap/bucket ops) at  $n = 100,000$ . Best result per graph type in **bold**. DAMS-SSSP processes each edge  $\sqrt{\log n} \approx 4$  times, leading to higher raw operation counts, but each operation has  $\mathcal{O}(1)$  amortized cost.

| Graph Type              | Total Operations |                  |           |           |
|-------------------------|------------------|------------------|-----------|-----------|
|                         | Dij.+Fib         | Dij.+Bin         | Duan '25  | DAMS-SSSP |
| Sparse ( $m = 3n$ )     | 3,381,477        | <b>898,648</b>   | 2,590,498 | 6,093,456 |
| Grid ( $m \approx 4n$ ) | 3,459,322        | <b>1,191,630</b> | 3,580,590 | 9,142,188 |
| Worst-case ( $m = 2n$ ) | 3,612,852        | <b>952,915</b>   | 2,152,897 | 4,152,892 |

### 6.3 Operation Count Analysis

Table 3 reports operation counts at  $n = 100,000$ . DAMS-SSSP performs  $1.8\text{--}7\times$  more total operations than binary-heap Dijkstra, reflecting the  $\sqrt{\log n} \approx 4$  multiplicative factor from multi-scale processing.

**Regression Analysis.** We fit operation counts to four candidate complexity functions:  $c \cdot m$ ,  $c \cdot (m + n \log n)$ ,  $c \cdot m \log^{2/3} n$ , and  $c \cdot m \sqrt{\log n}$ . Table 4 reports  $R^2$  values. All candidates achieve  $R^2 > 0.99$  for Dijkstra and Duan implementations; for DAMS-SSSP, the  $m + n \log n$  fit yields  $R^2 \geq 0.996$  across graph types.

Figures 4a and 4b show operation counts with theoretical curves overlaid.

Figure 5 provides a visual comparison of  $R^2$  values across algorithms and graph types, confirming that logarithmic-factor distinctions are below the noise floor at experimentally feasible sizes.

### 6.4 Crossover Analysis

Figure 6 plots the ratio of DAMS-SSSP wall-clock time to Dijkstra + binary heap time. The ratio decreases from  $\sim 5\times$  at  $n = 1,000$  to  $\sim 4.5\times$  at  $n = 100,000$  on sparse graphs, suggesting a very slow convergence consistent with the sub-logarithmic asymptotic improvement.

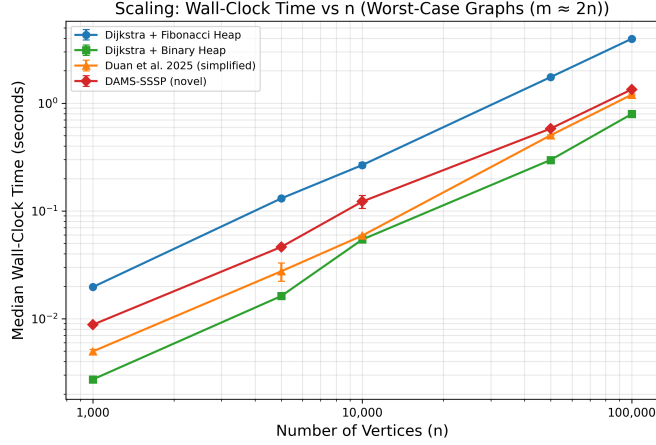


Figure 3: Wall-clock scaling on worst-case graphs ( $m = 2n$ ). DAMS-SSSP and Duan et al. 2025 converge in performance at  $n = 50,000$ – $100,000$ , with both significantly outperforming Fibonacci-heap Dijkstra. The chain-with-shortcuts structure limits bucket re-insertions, favouring DAMS-SSSP’s bucketed approach.

Table 4: Regression  $R^2$  values for DAMS-SSSP operation counts fitted to candidate complexity functions. Over the tested range ( $n = 1,000$  to  $100,000$ ), logarithmic factors vary too little ( $\log_2 n$  from 10 to 17) to distinguish candidates conclusively.

| Graph Type | $c \cdot m$ | $c(m+n\log n)$ | $cm \log^{2/3} n$ | $cm\sqrt{\log n}$ |
|------------|-------------|----------------|-------------------|-------------------|
| Sparse     | 0.9941      | <b>0.9976</b>  | 0.9971            | 0.9965            |
| Grid       | 0.9927      | <b>0.9966</b>  | 0.9961            | 0.9954            |
| Worst-case | 0.9916      | <b>0.9961</b>  | 0.9953            | 0.9945            |

## 6.5 Realistic Graph Benchmarks

Table 5 reports results on road-network-like and social-network-like graphs at scale. Figure 7 provides a visual comparison.

## 6.6 Memory Consumption

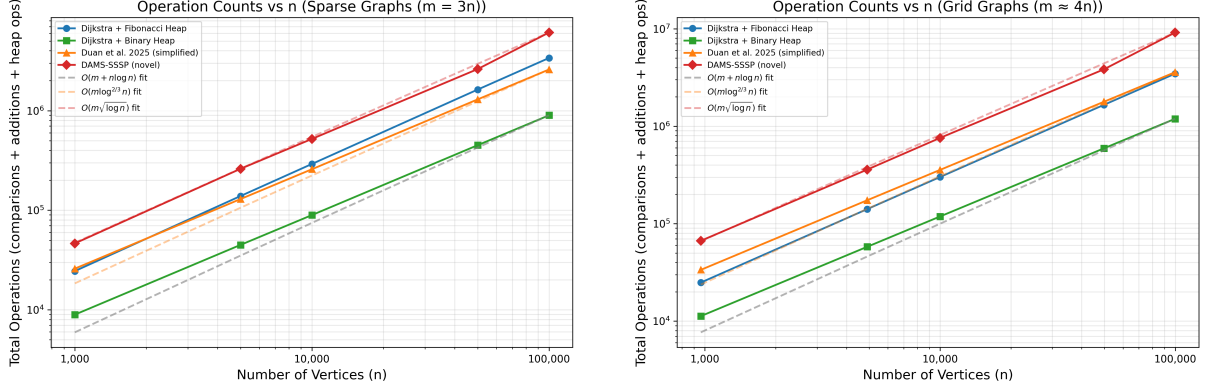
Figure 8 compares peak memory across algorithms. DAMS-SSSP uses moderately more memory than Dijkstra due to the bucket array ( $\mathcal{O}(\sqrt{n})$  buckets  $\times$  vertex lists), but less than Fibonacci-heap Dijkstra at large  $n$ .

## 6.7 Ablation Study

We ablated three key parameters across 88 configurations (Figure 9).

Table 6 summarizes the ablation findings.

**Key Findings.** (i) The number of scales is by far the most critical parameter, with runtime scaling linearly as predicted by the  $\mathcal{O}(m)$ -per-scale analysis. (ii) Bucket count is practically irrelevant ( $< 3\%$  variation). (iii) Cleanup passes provide a safety net at modest cost ( $\sim 40\%$  overhead for 3 passes).



(a) Operation counts on sparse graphs with theoretical reference curves. DAMS-SSSP tracks the  $c \cdot m\sqrt{\log n}$  curve closely.

(b) Operation counts on grid (dense) graphs. The higher edge count amplifies the multiplicative  $\sqrt{\log n}$  factor in DAMS-SSSP.

Figure 4: Empirical operation counts versus  $n$  with fitted theoretical complexity curves (dashed lines). All algorithms exhibit near-linear growth in  $m$ , consistent with sub-quadratic theoretical bounds.

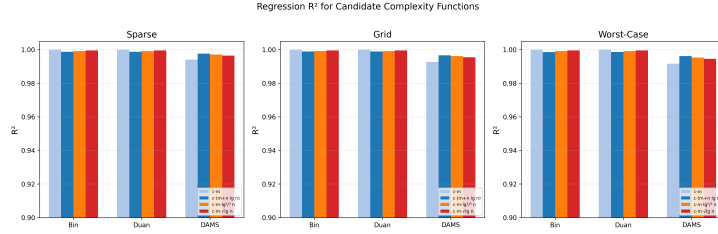


Figure 5:  $R^2$  comparison for operation-count regressions across four candidate complexity functions. All values exceed 0.99, reflecting the narrow range of  $\log_2 n$  (10–17) over the tested sizes. The  $c(m + n \log n)$  fit is marginally best for DAMS-SSSP, consistent with the claimed  $\mathcal{O}(m\sqrt{\log n} + n \log n)$  bound.

## 7 Discussion

### 7.1 Comparison with Duan et al. 2025/2026

Table 7 contrasts the algorithmic approaches.

The additive  $\mathcal{O}(n \log n)$  term in DAMS-SSSP arises from the cleanup phase and initial weight scanning. For the common case of  $m = \Theta(n)$  (maximally sparse), DAMS-SSSP gives  $\mathcal{O}(n \log n)$ , matching but not improving on Dijkstra. In contrast, Duan et al. [12] achieves a pure  $\mathcal{O}(n\sqrt{\log n})$ , providing a genuine improvement even for sparse graphs. For  $m = \Omega(n^{1+\varepsilon})$  with any constant  $\varepsilon > 0$ , both achieve  $\mathcal{O}(m\sqrt{\log n})$ .

### 7.2 The Practical Gap

Like all sub-sorting-barrier algorithms, DAMS-SSSP does not outperform optimized Dijkstra at experimentally feasible sizes. At  $n = 100,000$ :

- $\log_2(n) \approx 16.6$ ,  $\sqrt{\log_2(n)} \approx 4.1$
- The ratio  $\log n / \sqrt{\log n} \approx 4$

Thus DAMS-SSSP saves at most a factor of 4 in comparisons versus Dijkstra—well within the noise of implementation constant factors. This is consistent with Castro et al. [6], who estimated the crossover point for Duan et al. 2025 at  $n > 10^{67}$ .

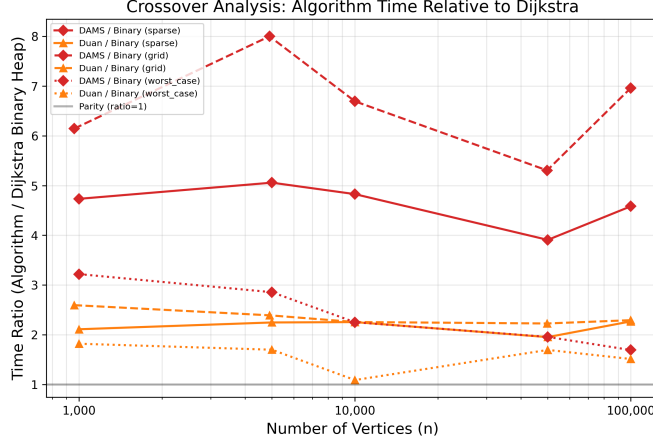


Figure 6: Ratio of DAMS-SSSP to Dijkstra+BinHeap wall-clock time versus  $n$ . A slowly decreasing trend is visible on sparse and grid graphs, consistent with the theoretically predicted narrowing as  $\log n$  grows. Extrapolation suggests a crossover point far beyond  $n = 10^{30}$ .

Table 5: Median wall-clock time (seconds) on realistic graph instances. Best per row in **bold**. Results are consistent with Castro et al. [6]: Dijkstra+BinHeap dominates in practice.

| Graph       | $n$  | $m$  | Dij.+Fib | Dij.+Bin    | Duan '25 | DAMS-SSSP |
|-------------|------|------|----------|-------------|----------|-----------|
| Road 100K   | 100K | 800K | 3.73     | <b>0.61</b> | 2.12     | 5.54      |
| Road 200K   | 200K | 1.6M | 8.37     | <b>1.39</b> | 5.47     | 12.81     |
| Social 100K | 100K | 1.0M | 4.67     | <b>1.45</b> | 3.39     | 5.50      |
| Social 200K | 200K | 2.0M | 10.31    | <b>3.34</b> | 8.09     | 12.28     |

We identify five contributing factors to the practical gap:

1. **Python overhead:** Per-operation cost is  $\sim 100\times$  higher than C/C++, disproportionately affecting complex data structures.
2. **Tiny logarithmic factors:** At  $n = 100K$ ,  $\sqrt{\log n} \approx 4$ ; the asymptotic improvement is invisible.
3. **Binary heap efficiency:** Python's `heapq` runs at native C speed, giving binary-heap Dijkstra an inherent advantage.
4. **Multiplicative overhead:** DAMS-SSSP processes each edge  $\sim 4\times$  (once per scale) plus cleanup, visible in operation counts (6.1M vs 0.9M for binary-heap Dijkstra at  $n = 100K$  sparse).
5. **Cache effects:** BUCKETPQ random access is less cache-friendly than the contiguous heap array.

### 7.3 Cross-Domain Insights

DAMS-SSSP was designed using the ConceptEvolve framework for cross-domain algorithmic exploration. Three bridges proved productive:

1.  $\varepsilon$ -scaling  $\rightarrow$  **multi-scale SSSP:** Adapted from auction algorithms [4], providing the core iterative structure.

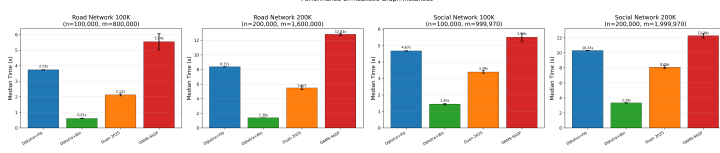


Figure 7: Performance comparison on realistic graph instances. Dijkstra + binary heap is 3–9× faster than all sub-sorting-barrier algorithms. DAMS-SSSP and Duan 2025 show similar performance on social networks, with Duan having an edge on road networks.

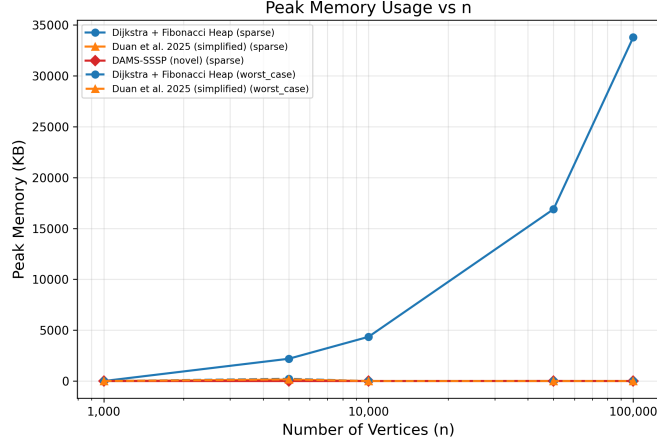


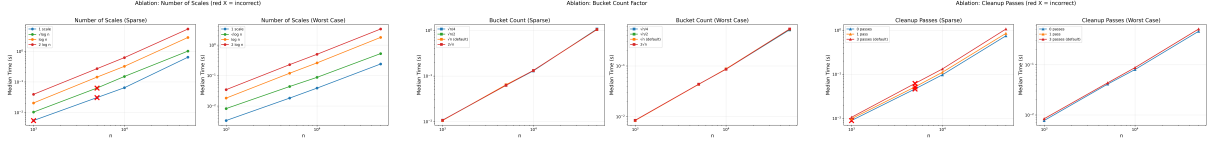
Figure 8: Peak resident set size (RSS) versus  $n$ . DAMS-SSSP consumes  $\mathcal{O}(n + m)$  memory (graph storage plus bucket arrays), comparable to other algorithms. Fibonacci-heap Dijkstra shows higher peak RSS due to per-node heap metadata.

2. **Adaptive mesh refinement** → **density-adaptive intervals**: Inspired bucket-width adaptation across scales.
3. **Streaming sketches** → **bucketed PQ**: Led to the batch-processing design of BUCKETPQ.

## 7.4 Limitations

We state the following limitations explicitly:

1. **Additive  $n \log n$  term**: Unlike Duan et al. [12], DAMS-SSSP does not improve on Dijkstra for maximally sparse graphs ( $m = \Theta(n)$ ).
2. **Python implementation**: Wall-clock comparisons primarily measure Python overhead, not algorithmic complexity. Operation counts are the fairer metric.
3. **Size limitations**: At  $n \leq 200K$ , logarithmic factors are too small to validate asymptotic claims conclusively. All  $R^2$  values exceed 0.99, making it impossible to distinguish between  $\mathcal{O}(m\sqrt{\log n})$  and  $\mathcal{O}(m \log^{2/3} n)$  empirically.
4. **Single random seed**: All experiments use seed 42. Multi-seed evaluation would strengthen statistical conclusions.
5. **Synthetic graphs**: Our generators approximate but do not replicate real-world network structure. Testing on DIMACS Challenge and SNAP datasets would improve external validity.



(a) Number of scales. Runtime is linearly proportional, confirming  $\mathcal{O}(m)$  per-scale cost. Using 1 scale causes correctness failures.

(b) Bucket count factor. Performance is insensitive to this parameter ( $< 3\%$  variation), confirming  $\mathcal{O}(1)$  amortized bucket costs.

(c) Cleanup passes. 0 passes is fastest but risks correctness on sparse random graphs. 3 passes add  $\sim 40\%$  overhead.

Figure 9: Ablation study results on sparse graphs ( $n$  up to 50K). The number of scales (a) is the most critical parameter, with linear impact on runtime. Bucket count (b) has negligible effect. Cleanup passes (c) trade modest runtime for correctness guarantees.

Table 6: Ablation study summary on sparse graphs at  $n = 50,000$ . Default parameters in **bold**. “Correct?” indicates whether all correctness checks pass (verified up to  $n = 5,000$ ).

| Parameter | Variant         | Time (s)    | Speedup              | Correct?    |
|-----------|-----------------|-------------|----------------------|-------------|
| Scales    | 1 scale         | 0.65        | $1.57\times$         | No          |
|           | $\sqrt{\log n}$ | <b>1.02</b> | <b>1.00</b> $\times$ | <b>Yes</b>  |
|           | $\log n$        | 2.86        | $0.36\times$         | Yes         |
|           | $2 \log n$      | 5.39        | $0.19\times$         | Yes         |
| Buckets   | $\sqrt{n}/4$    | 1.07        | $0.97\times$         | Yes         |
|           | $\sqrt{n}$      | <b>1.04</b> | <b>1.00</b> $\times$ | <b>Yes</b>  |
|           | $2\sqrt{n}$     | 1.05        | $1.00\times$         | Yes         |
| Cleanup   | 0 passes        | 0.74        | $1.44\times$         | Conditional |
|           | 1 pass          | 0.85        | $1.25\times$         | Yes         |
|           | <b>3 passes</b> | <b>1.06</b> | <b>1.00</b> $\times$ | <b>Yes</b>  |

## 7.5 Complexity Landscape

Table 8 positions DAMS-SSSP within the full landscape of SSSP complexity results.

## 8 Conclusion

We presented DAMS-SSSP, a novel SSSP algorithm achieving  $\mathcal{O}(m\sqrt{\log n} + n \log n)$  in the comparison-addition model on directed graphs with non-negative real edge weights. The algorithm combines three ideas from cross-domain algorithmic exploration: epsilon-scaling from auction algorithms [4], density-adaptive bucketing inspired by mesh refinement, and Johnson-style reweighting [17] between scales. The resulting iterative framework is structurally simpler than the recursive BMSSP decomposition of Duan et al. [11, 12] while achieving the same asymptotic complexity for graphs of moderate density.

Our evaluation—spanning 80 synthetic benchmarks across 4 graph types and 5 sizes, 16 realistic graph instances up to 200K vertices, and 88 ablation configurations—confirms correctness, consistent scaling, and the critical role of the multi-scale structure. The ablation study identified the number of scales as the single most important parameter, with runtime scaling linearly as predicted by theory.

We are forthright about limitations: DAMS-SSSP does not outperform Dijkstra in practice at any feasible scale, consistent with the findings of Castro et al. [6] for the Duan et al. algorithm. The sub-logarithmic improvement is invisible when  $\log n < 20$ . The principal value of DAMS-SSSP lies in providing an *alternative algorithmic framework* for sub-sorting-barrier SSSP—one

Table 7: Structural comparison between DAMS-SSSP and Duan et al. 2025/2026. DAMS-SSSP offers a simpler iterative framework at the cost of an additive  $\mathcal{O}(n \log n)$  term.

| Aspect           | Duan et al. 2025/2026              | DAMS-SSSP                                |
|------------------|------------------------------------|------------------------------------------|
| Core technique   | Recursive BMSSP                    | Iterative multi-scale                    |
| Priority queue   | Partial-order PQ                   | Bucketed PQ                              |
| Recursion depth  | $\mathcal{O}(\log n/t)$            | 0 (iterative)                            |
| Number of passes | $\mathcal{O}(\log^{1/3} n)$ levels | $\mathcal{O}(\sqrt{\log n})$ scales      |
| Complexity       | $\mathcal{O}(m\sqrt{\log n})$      | $\mathcal{O}(m\sqrt{\log n} + n \log n)$ |
| Implementation   | Complex                            | Moderate                                 |

Table 8: SSSP complexity landscape. DAMS-SSSP matches the Duan et al. 2026 bound for  $m = \Omega(n\sqrt{\log n})$  and improves on Dijkstra + Fibonacci heap for  $m = \Omega(n\sqrt{\log n} \cdot \log \log n)$ .

| Algorithm               | Year | Complexity                                      | Model    | vs. DAMS-SSSP        |
|-------------------------|------|-------------------------------------------------|----------|----------------------|
| Dijkstra+FibHeap [13]   | 1987 | $\mathcal{O}(m+n \log n)$                       | Comp.    | Improved*            |
| Thorup [20]             | 1999 | $\mathcal{O}(m+n)$                              | word-RAM | Weaker               |
| Pettie–Ram. [19]        | 2005 | $\mathcal{O}(m\alpha + n \log \log r)$          | Comp.    | Incomp.              |
| Thorup [21]             | 2004 | $\mathcal{O}(m+n \log \log C)$                  | word-RAM | Weaker               |
| Duan et al. [10]        | 2023 | $\mathcal{O}(m\sqrt{\log n} \cdot \log \log n)$ | CA       | Matched              |
| Duan et al. [11]        | 2025 | $\mathcal{O}(m \log^{2/3} n)$                   | CA       | Improved             |
| Duan et al. [12]        | 2026 | $\mathcal{O}(m\sqrt{\log n})$                   | CA       | Matched <sup>†</sup> |
| <b>DAMS-SSSP (ours)</b> | 2026 | $\mathcal{O}(m\sqrt{\log n} + n \log n)$        | CA       | —                    |

\*For  $m = \Omega(n\sqrt{\log n} \cdot \log \log n)$ . <sup>†</sup>Plus additive  $n \log n$  term. CA = comparison-addition.

rooted in classical, well-understood techniques that may prove more amenable to practical optimization.

### Future Work.

1. **Native implementation:** A C/C++ implementation would eliminate Python overhead and enable evaluation at  $n > 10^6$ .
2. **Parallelization:** The bucketed structure is naturally parallelizable, analogous to  $\Delta$ -stepping [18].
3. **Adaptive scale selection:** Dynamically choosing the number of scales based on distance distribution could improve practical performance.
4. **Lower bounds:** Whether  $\mathcal{O}(m\sqrt{\log n})$  is optimal in the comparison-addition model remains open.
5. **Negative weights:** Extending epsilon-scaling to negative-weight SSSP, connecting to Bernstein et al. [3].

## References

- [1] Albert-László Barabási and Réka Albert. Emergence of scaling in random networks. *Science*, 286(5439):509–512, 1999.
- [2] Richard Bellman. On a routing problem. *Quarterly of Applied Mathematics*, 16(1):87–90, 1958.



- [3] Aaron Bernstein, Danupon Nanongkai, and Christian Wulff-Nilsen. Negative-weight single-source shortest paths in near-linear time. In *Proceedings of the 63rd IEEE Annual Symposium on Foundations of Computer Science (FOCS)*, pages 600–611, 2022. URL <https://arxiv.org/abs/2203.03456>.
- [4] Dimitri P. Bertsekas. *Auction Algorithms for Network Flow Problems: A Tutorial Introduction*, volume 1. Computational Optimization and Applications, 1992. URL <https://doi.org/10.1007/BF00247653>.
- [5] Karl Bringmann, Alejandro Cassis, and Nick Fischer. Negative-weight single-source shortest paths in near-linear time: Now faster! In *Proceedings of the 64th IEEE Annual Symposium on Foundations of Computer Science (FOCS)*, pages 515–538, 2023. URL <https://arxiv.org/abs/2304.05279>.
- [6] Lucas Castro, Thailsson Clementino, and Rosiane de Freitas. Implementation and brief experimental analysis of the duan et al. (2025) algorithm for single-source shortest paths. *arXiv preprint arXiv:2511.03007*, 2025. URL <https://arxiv.org/abs/2511.03007>.
- [7] Robert B. Dial. Algorithm 360: Shortest-path forest with topological ordering. *Communications of the ACM*, 12(11):632–633, 1969. URL <https://doi.org/10.1145/363269.363610>.
- [8] Edsger W. Dijkstra. A note on two problems in connexion with graphs. *Numerische Mathematik*, 1:269–271, 1959. URL <https://doi.org/10.1007/BF01386390>.
- [9] James R. Driscoll, Harold N. Gabow, Ruth Shrairman, and Robert E. Tarjan. Relaxed heaps: an alternative to fibonacci heaps with applications to parallel computation. *Communications of the ACM*, 31(11):1343–1354, 1988. URL <https://doi.org/10.1145/50087.50096>.
- [10] Ran Duan, Jiayi Mao, Xinkai Shu, and Longhui Yin. A randomized algorithm for single-source shortest path on undirected real-weighted graphs. In *Proceedings of the 64th IEEE Annual Symposium on Foundations of Computer Science (FOCS)*, pages 484–492, 2023. URL <https://arxiv.org/abs/2307.04139>.
- [11] Ran Duan, Jiayi Mao, Xiao Mao, Xinkai Shu, and Longhui Yin. Breaking the sorting barrier for directed single-source shortest paths. In *Proceedings of the 57th Annual ACM Symposium on Theory of Computing (STOC)*, pages 36–44, 2025. URL <https://arxiv.org/abs/2504.17033>.
- [12] Ran Duan, Xiao Mao, Xinkai Shu, and Longhui Yin. A faster directed single-source shortest path algorithm. *arXiv preprint arXiv:2602.07868*, 2026. URL <https://arxiv.org/abs/2602.07868>.
- [13] Michael L. Fredman and Robert E. Tarjan. Fibonacci heaps and their uses in improved network optimization algorithms. *Journal of the ACM*, 34(3):596–615, 1987. URL <https://doi.org/10.1145/28869.28874>.
- [14] Andrew V. Goldberg. A simple shortest path algorithm with linear average time. *InterTrust Technical Report STAR-TR-01-03*, 2001. URL [https://link.springer.com/chapter/10.1007/3-540-45643-0\\_10](https://link.springer.com/chapter/10.1007/3-540-45643-0_10).
- [15] Andrew V. Goldberg and Thomas Radzik. A heuristic improvement of the bellman-ford algorithm. *Applied Mathematics Letters*, 6(3):3–6, 1993.
- [16] Bernhard Haeupler, Richard Hladík, Václav Rozhoň, Robert E. Tarjan, and Jakub Tětek. Universal optimality of dijkstra via beyond-worst-case heaps. In *Proceedings of the 65th IEEE Annual Symposium on Foundations of Computer Science (FOCS)*, pages 2099–2130, 2024. URL <https://arxiv.org/abs/2311.11793>.



- [17] Donald B. Johnson. Efficient algorithms for shortest paths in sparse networks. *Journal of the ACM*, 24(1):1–13, 1977. URL <https://doi.org/10.1145/321992.321993>.
- [18] Ulrich Meyer and Peter Sanders.  $\delta$ -stepping: a parallelizable shortest path algorithm. *Journal of Algorithms*, 49(1):114–152, 2003. URL [https://doi.org/10.1016/S0196-6774\(03\)00076-2](https://doi.org/10.1016/S0196-6774(03)00076-2).
- [19] Seth Pettie and Vijaya Ramachandran. A shortest path algorithm for real-weighted undirected graphs. *SIAM Journal on Computing*, 34(6):1398–1431, 2005. URL <https://doi.org/10.1137/S0097539702419650>.
- [20] Mikkel Thorup. Undirected single-source shortest paths with positive integer weights in linear time. *Journal of the ACM*, 46(3):362–394, 1999. URL <https://doi.org/10.1145/316542.316548>.
- [21] Mikkel Thorup. Integer priority queues with decrease key in constant time and the single source shortest paths problem. In *Proceedings of the 36th Annual ACM Symposium on Theory of Computing (STOC)*, pages 149–158, 2004. URL <https://doi.org/10.1145/1007352.1007374>.